

AUTOCON5 · WS:C2 · VEXPERTAI

From Alert to Action

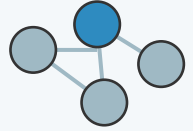
Building a 5-Agent AI System That Cuts MTTR

Eduard Dulharu · live on a multi-vendor EVPN-VXLAN fabric (Cisco NX9K + Arista vEOS)

AUTOCON5 2026 · [networkautomation.forum](https://networkautomationforum.com)



How we'll spend our four hours

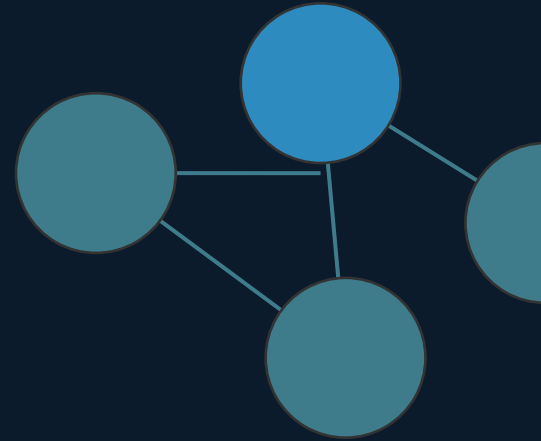


- ~45 min — the ideas + the architecture (just enough to follow everything after).
- ~50 min — LIVE on a real multi-vendor fabric: the agents diagnose, the digital twin rehearses the fix, data flowing through the containers in front of you.
- ~15 min — break.
- ~95 min — HANDS-ON on YOUR own pod: observe a fault → read the agents' minds → BUILD your own specialist → extend the catalogue with your name on it.
- The rest is yours — Q&A and one-to-one time. The full deck is your take-home.

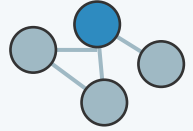
★ Mostly hands-on, on purpose: you'll leave having BUILT a piece of this, not just watched it.

PART 0

Why we're here



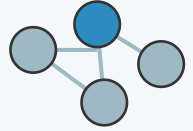
The 2 a.m. page you've answered a hundred times



- The alert fires. You're awake before you've read it — because you already know roughly what it is.
- A BGP-EVPN session went IDLE again. A VNI doesn't line up across two vendors. A route-target's off by one digit. Not the hard ones — the boring repeat offenders.
- You SSH in half-asleep, run the same five show commands, find the one wrong line, fix it, go back to bed.
- The fault was never the problem. The problem is that it's you, at 2 a.m., again.

★ The expensive faults aren't the rare ones — they're the familiar ones that keep coming back.

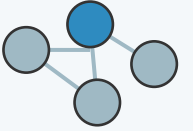
What it actually costs



- MTTR on a familiar cross-vendor fault is rarely minutes — it's a war-room afternoon: open a bridge, page routing, page the EVPN person, page the vendor specialist.
- Real bring-up example: a route-target reading 65003:10011 where every other leaf said 10010. One character. Ninety minutes before anyone spotted it.
- Attach a number: illustratively, a trading-floor tenant at ~\$1.25M/hr turns 'two hours of triage' into a six-figure event.
- The quieter cost: three senior people pulled into a bridge for something one of them could've fixed alone.

★ Two hours of war-room on a one-character bug. That math is the whole reason we're here.

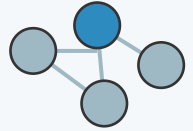
The real problem: the fix lives in three people's heads



- The knowledge that solves the 2 a.m. page isn't in a runbook — it's in the muscle memory of two or three senior engineers.
- When they're asleep, on holiday, in another incident, or interviewing elsewhere — MTTR isn't 100 seconds, it's 'whenever we can reach them.'
- **Tribal knowledge is a single point of failure** you can't monitor, can't alert on, can't put in a maintenance window.
- Turnover and burnout don't just cost a hire — they cost the only working copy of how your fabric actually gets fixed.

★ Your most critical diagnostic system is three undocumented humans. Until now.

Four problems, named plainly

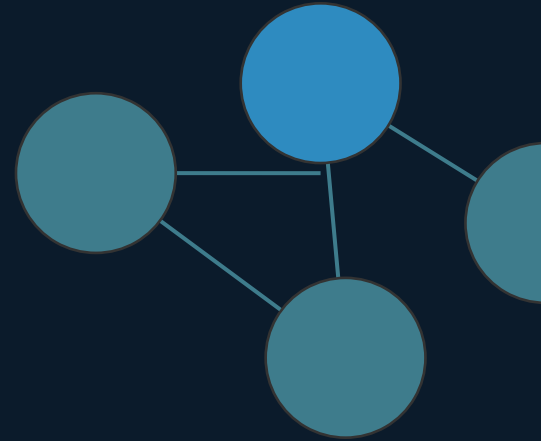


- (a) **Time-to-resolve** — collapse familiar-fault MTTR from a war-room afternoon toward seconds, with a verdict and the evidence behind it.
- (b) **On-call burnout / tribal-knowledge capture** — get the senior engineers' diagnostic steps out of their heads into a system that runs them deterministically, every time.
- (c) **Change risk** — catch the bad config (the off-by-one VNI, the telnet line) before it reaches production, not in the post-mortem.
- (d) **Compliance drift** — know which devices have silently fallen out of policy, continuously, instead of at audit time.

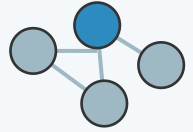
★ Four ordinary operational problems. No moonshots — just the boring ones that hurt.

THE BIGGER IDEA

Institutional Memory as Code



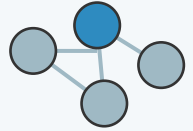
It was never four problems — it's one, with three symptoms



- Every team has a “Dana” — eleven years in, who just KNOWS the 2 a.m. spine flap is usually a transceiver, not the route policy.
- Dana isn't the problem. Dana being the only copy is — a single point of failure you can't monitor, can't back up, can't scale.
- MTTR, change risk and compliance drift aren't separate problems — they're what happens when the fix lives in three people's heads.
- Capture the knowledge and all three improve at once — because you fixed the cause, not the symptoms.

★ Tribal knowledge is the root cause. MTTR, change risk and compliance drift are just its symptoms.

The catalogue is the brain. The agents are the runtime.

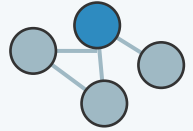


Analogy: *GitOps — but for the knowledge inside your engineers' heads. Know-how becomes versioned, peer-reviewed, executable YAML.*

- The brain = your catalogue: fault scenarios, diagnostic trees, remediation playbooks and policy — already ~76 scenarios + 42 diagnostic trees of real YAML.
- The five agents don't 'know' anything — they EXECUTE the brain, deterministically, 24/7.
- Seniors write the brain once; it runs forever — asleep, on holiday, or gone.
- Data, not code: a new fault is a new YAML entry, reviewed like a pull request — no redeploy.

★ This is Institutional Memory as Code — your seniors' muscle-memory, turned into an institutional asset.

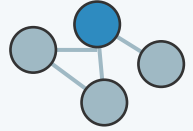
Knowledge becomes a pull request — in six steps



- 1 • CAPTURE — a senior resolves an incident; the system drafts a catalogue entry from the reasoning chain (AI-assisted).
- 2 • AUTHOR — catalogue-cli new-scenario scaffolds the YAML: trigger • diagnostic tree • remediation playbook • policy • author.
- 3 • VALIDATE + TEST — schema check + a fixture proving it routes and diagnoses correctly.
- 4 • REVIEW — a peer approves the pull request (knowledge governed like code).
- 5 • CI GATE — validate + fixtures + pattern-ID sync run automatically; a bad entry can't merge.
- 6 • MERGE + HOT-LOAD — the agents acquire the new knowledge live, no redeploy.

★ This IS today's Lab 3 — you'll author a scenario and open a PR, live, in 45 minutes.

The brain becomes SKILLS — tailored to YOUR environment



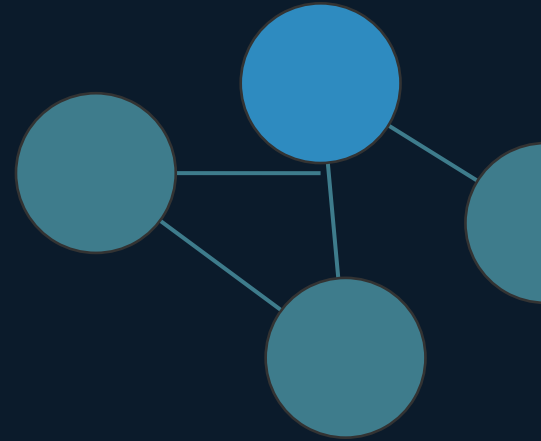
Analogy: *Each knowledge entry is a SKILL the agents acquire — like apps for a phone, but for network operations and specific to your fabric.*

- A Skill = one self-contained unit of know-how: when it applies + how to diagnose it + how to fix it + the guardrails, with provenance.
- Every environment gets its OWN skill set: your vendors, your tenants, your \$1.25M/hr trading SLA, your PCI freeze window — not a generic template.
- Universal skills (BGP-EVPN basics) ship out of the box; env-specific skills you author once and own forever.
- Skills are shareable + forkable — a community registry (Apache-2.0): an ACI pack, an SD-WAN pack, a vendor's own pack.
- Your environment's skill set IS the moat: unique, compounding, peer-reviewed — competitors can buy models, not your ten years of muscle-memory.

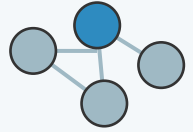
★ One SSOT → many Skills → a brain custom to every environment. The library grows with every incident.

PART 1 · NO AI BACKGROUND NEEDED

Seven ideas, ninety seconds each



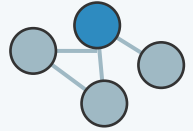
The LLM: a very well-read junior who's never seen YOUR network



Analogy: *A new hire who's memorised every RFC and vendor guide ever written — but has never logged into a single one of your boxes.*

- It's a text-prediction engine: give it words, it writes the most plausible next words. That's all.
- Strength: it has read the entire internet — it 'knows' BGP, EVPN, VXLAN, design patterns, in general.
- Blind spot: zero live facts about YOUR fabric. Ask it your leaf-2's VNI and it will confidently make one up — that's a hallucination, not a lookup.
- By itself it can only talk. It can't run a show command, read a switch, or change anything.

★ Every 'reasoning' step today is one of these — but we never let it answer your network from memory; we force-feed it real device facts first.



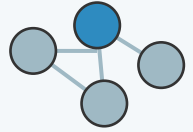
The AI Agent: that junior, but now it can actually DO things

Analogy: Same junior — except now they have lab access, a runbook, and permission to run show commands and propose a fix, not just talk.

- An agent = an LLM wrapped in a loop: observe → decide which tool to call → call it → look at the result → decide again — until the job's done.
- It has a **goal** ('triage this alert') and a **toolbox** (read the device, query the graph, rehearse a fix) — not just a chat window.
- It chains steps: read the switch, notice PfxRcd=0, form a hypothesis, propose a config — like a real engineer working a ticket.
- Crucially, it stops and asks a human before anything risky touches production.

★ Today's system is FIVE of these — a Team Leader plus four specialists (Stability, Troubleshooting, Security, Architect).

RAG: turn the closed-book exam into an open-book one

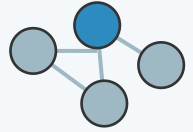


Analogy: *Don't let the junior answer from memory — make it an open-book exam: a librarian fetches the exact right pages first, THEN they answer from those pages.*

- RAG = Retrieval-Augmented Generation. Plain words: fetch the right reference first, then let the model write — grounded in what you fetched.
- We retrieve the actual relevant docs (design cards, RFC excerpts, live fabric facts) and hand them over with the question.
- The answer must cite what we fed it. If nothing relevant comes back, the agent abstains — 'I won't invent a source' — instead of bluffing.
- Knowledge lives in editable files, not baked into the model's weights — add a doc, it's instantly usable, no retraining.

★ In Demo 4 the Architect answers a design question with a 'Sources:' footer — and when you ask it to bake sourdough, it honestly declines.

MCP: the agent's hands — a standard plug for calling real tools

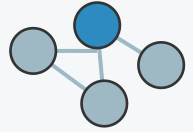


Analogy: Think USB-C for AI. One standard plug that lets any agent call a real tool or read a real device — instead of every agent hand-wiring its own connector.

- MCP = Model Context Protocol. Plain words: a standard way for an agent **to call tools** — 'read this switch's BGP-EVPN state', 'scan this device for CVEs'.
- It's the agent's HANDS. The agent says 'read leaf-2'; MCP is the standard plug it calls — and the MCP server's tool performs the read against the real box (via the vendor adapter).
- One standard plug = a tool you write once works for every agent — no bespoke glue per agent, per device.
- It's the controlled chokepoint: every device touch goes through here, so it can be read-only, logged and rate-limited.

★ Every device fact flows through the MCP server on :8001 — agents NEVER SSH a switch directly. 'source=adapter:N9kvAdapter' is your proof a real read happened.

A2A: agents passing the ticket between specialists

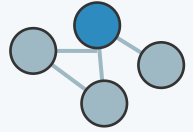


Analogy: A NOC bridge in software: the NOC lead triages, hands the ticket to the routing SME, who hands it to the security SME — each does their part and passes it on.

- A2A = Agent-to-Agent. Plain words: a common **message format** so one agent can hand work to another and get a structured answer back.
- It's how the Team Leader delegates: 'this is an EVPN route-target problem → you, Stability, take it.'
- Messages are typed — delegate / return result / escalate / status — so the handoff is clean, not a free-text chat.
- Lets you specialise: a routing brain, a security brain, an interface brain — each focused, all talking **one shared protocol**.

★ When the fault fires in Act 4, A2A is the Team Leader routing it to the right specialist in ~3 seconds.

Knowledge Graph: your fabric as dots and lines you can query

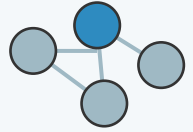


Analogy: *A live picture of your network as dots and lines — every device, interface, VNI, BGP session is a dot; every neighbor and session is a line you can walk along.*

- A graph is just dots (**nodes**) and lines (**relationships**). A dot is a device/interface/VNI/route; a line is 'is a neighbor of', 'has this session'.
- We store it in a graph database, Neo4j — you can open a browser and literally see the dots and lines.
- Because it's all connected, you ask relationship questions instantly: 'can leaf-nx reach leaf-ar?', 'who's affected if this prefix dies?'
- One shared, fast map every agent reads — so they don't each hammer your routers to relearn the same topology.

★ The grounded reachability answers come straight from here — 'can leaf-nx reach leaf-ar?' returns in 0.01s, graph-truth, zero hallucination.

Digital Twin: a flight simulator for network changes

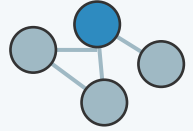


Analogy: *A flight simulator for your fabric — rehearse the proposed fix on a faithful copy and watch what breaks, before it ever touches the real switch.*

- A what-if sandbox — same topology, same sessions, but safe to experiment on.
- Before any fix is approved, the twin rehearses it: does reachability come back? what else moves? who gets hurt?
- It turns 'looks right to me' into 'tested' — the same instinct as a maintenance-window dry-run, but in seconds.
- It also computes the blast radius and dollar impact — which is what routes a risky change to a human instead of auto-applying.
- Two tiers: an instant graph-sim on every change, PLUS a full vendor twin running your EXACT production OS images for the changes that matter.

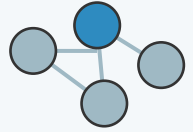
★ In the climax, once the specialist proposes the fix (confidence 0.95), the twin rehearses it and computes the ~\$1.675M/hr blast radius — THEN routes to a human.

The in-loop Digital Twin: a flight simulator built from the live graph



- Built from the live Knowledge Graph — the topology (devices + their adjacencies) loads into an in-memory **NetworkX graph**. NetworkX = an open-source Python graph library: devices are nodes, links are edges — a live 'map of the network in code' you can run what-if tests on. No devices touched, runs in milliseconds.
- Rehearses the fix — destructive commands remove the matching nodes/edges from the graph copy, then it re-runs reachability: 'will the fabric still converge after this change?'
- Scores command risk — each fix command is graded by rule (e.g. 'no router bgp' = high) → a single change-risk number.
- Computes blast radius — walks the graph from the affected prefix → devices, tenants, users, and \$ / hr exposed (the \$1.675M total on the card).
- Verdict — passes only if reachability holds AND change-risk < 0.75; otherwise it fails closed to a human.

★ `agents/shared/graph_simulator.py` • `GraphSimulator.simulate()`: reads the KG → NetworkX → change-risk + blast radius (+ reachability on an OSPF underlay), in milliseconds. On the BGP-EVPN fabric the decisive Tier-1 signals are risk + blast radius; full same-image reachability is re-validated on the Tier-2 vendor twin.



The high-fidelity twin: the SAME vendor OS as production

- The real aircraft, not a model — a **containerlab twin** on a bare-metal host boots the EXACT production images: Cisco NX-OS 9.3(6) and Arista EOS 4.31.10M, packaged from EVE-NG's own qcow2 via vrnetlab.
- A true mirror — the twin's wiring is rebuilt from LIVE LLDP and seeded with the real device configs, so BGP-EVPN actually converges .
- **Kept in sync** — a Config-as-Code reconciler pulls each device's running-config (git-versioned source of truth) and re-applies it to the twin every 10 minutes, with drift detection.
- Wired into the decision — when a fix clears Tier-1 and the confidence gate, **the agent REPRODUCES the fault**, applies the proposed fix, and verifies recovery on the real-image twin, before a human ever sees the card ('Mode: containerlab_live').
- Independently QA'd — a skeptic test changed the twin out-of-band; the service detected that exact state and restored it — proving it drives a real device, not a canned 'passed'.

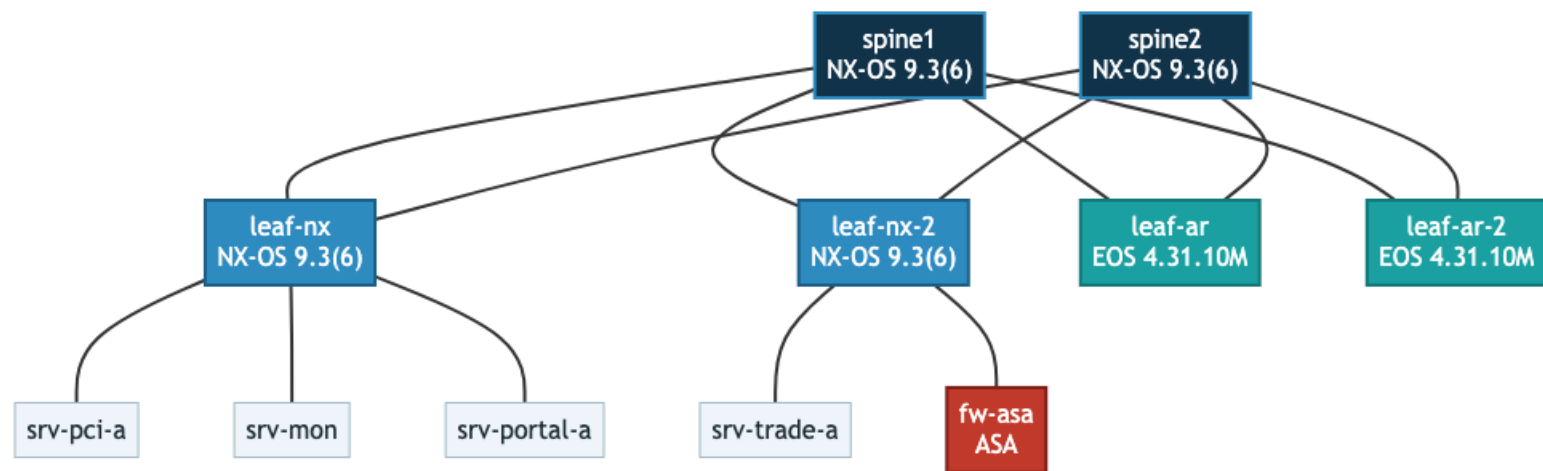
★ Proven live 2026-06-06: troubleshooting (interface) → 'Tier-2 metal twin verdict: passed • image NXOS 9.3(6) • reproduce-fix-verify'; stability (BGP peer) → twin peers 2→1→2. Envelope: NX-OS proven; Arista/EOS apply-path + the stability-BGP fix AS-number are post-workshop polish.

The 5-agent system — MCP-gated, KG-grounded, twin-validated



The Digital Twin — generated live from the running twin

Same vendor OS images as production · BGP-EVPN converged · config-synced every 10 min

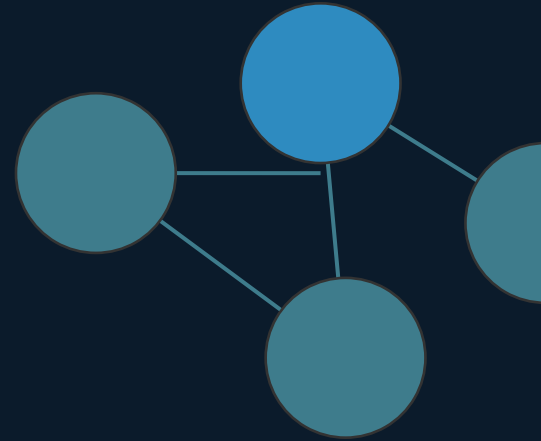


Legend: ■ Spine (NX-OS) ■ Leaf · Cisco NX-OS 9.3(6) ■ Leaf · Arista EOS 4.31.10M ■ Firewall (ASA) □ Host

Drawn from the **running twin**, not by hand — one command on the DT host: `dt-show` → the nodes, `show version` (= production), and BGP-EVPN **Established**.

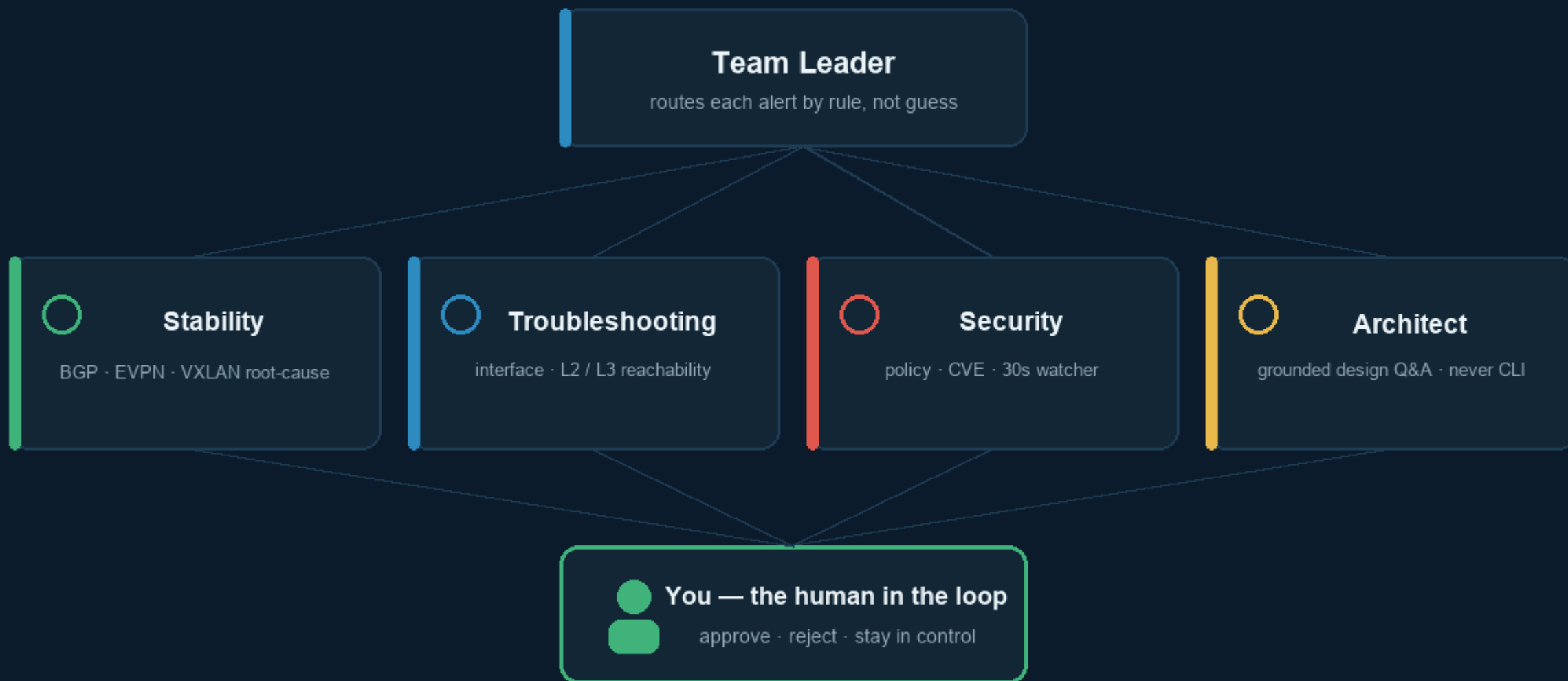
PART 2

The solution: a team of specialists, not a magic box

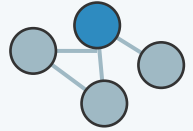


Your AI specialist team

Five brains — one human always in charge

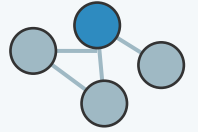


Five layers, five agents, one strict chain of command



- L1 Streamlit UI → L2 over ACP (REST/HTTP :8002) — the only way a human acts on the system
- L2 Team Leader → L3 specialists over A2A (Redis inbox queues) — one message, one specialist
- L3 specialists (Stability / Troubleshooting / Security / Virtual Architect) reach devices ONLY through MCP (:8001) — never SSH directly
- L4 MCP server is the single authorized gateway to L5 devices (SSH/NETCONF); every tool call carries an idempotency key
- Protocol assignment is fixed, not a suggestion: 1 ↔ 2 ACP, 2 ↔ 3 A2A, 3 ↔ 4 MCP — the layering is enforced by structure

★ An alert enters at L5 and climbs the same path every time — no agent skips a layer.



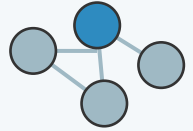
Four ways we ground the answer — and where RAG fits

- Catalogue (SSOT) — KNOWN faults: a deterministic YAML lookup (owner_specialist + the diagnosis). Exact, auditable, no hallucination. This is what the hands-on labs build.
- MCP tools — LIVE device facts: read the real switch right now (BGP-EVPN state, drops, CVEs). 'source=adapter' is the receipt a real read happened.
- Knowledge Graph (Neo4j) — RELATIONSHIPS & blast radius: 'can leaf-nx reach leaf-ar?', 'who's hurt if this prefix dies?' — graph-truth in ~0.01s.
- RAG — OPEN-ENDED design/advisory: retrieve the right knowledge cards / RFCs / CVEs, then generate with a 'Sources:' footer. This is the Virtual Architect and the CVE scanner — NOT the fault specialists.
- **The LLM only ever reasons over what these four hand it** — never from its own memory.

★ Right tool per question:

- a KNOWN fault → deterministic catalogue + graph (no guessing);
- an OPEN design question → RAG over a corpus. RAG lives in the Architect/scanner lane; the fault agents are grounded by catalogue + MCP + graph.

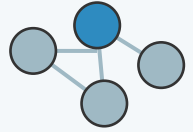
What each piece does — one line each



- Team Leader: drains the incident queue, deterministically triages, suppresses cascades, delegates to one specialist, assembles the human approval card
- The 4 specialists: **Stability** (BGP / EVPN / VXLAN) · **Troubleshooting** (interface / L2 / L3 reachability) · **Security** (ACL / auth / CVE) · **Virtual Architect** (design Q&A, never CLI)
- **MCP tools**: ~50 read-only-by-default device tools behind one authenticated gateway — the only thing that touches hardware
- **Neo4j Knowledge Graph**: the fabric's shared memory (topology, VNIs, VTEPs, tenants) for blast radius + reachability
- **LangChain RAG + Digital Twin** + audit chain: ground a render, rehearse a fix and hash-chain every decision for replayable audit

★ Specialists propose; the Team Leader and a human dispose.

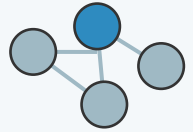
Alert to action: the data flow across three protocols



- Syslog → incident_queue → Team Leader picks it up
- Team Leader runs **deterministic regex triage** → classifies {agent, fault_type} → A2A delegation to ONE specialist
- Specialist gathers live evidence via MCP → runs its model → multi-pass validation (confidence → graph-sim → twin → policy → blast radius)
- Specialist returns a result → Team Leader builds an approval card
- Human approves over ACP → change-executor applies (idempotency, denylist, maintenance-window guards) → Post-Change Verifier diffs before/after

★ Three protocols, one direction of trust: each layer only speaks to its neighbor.

Self-hosted fine-tuned models — no public LLM sees your configs

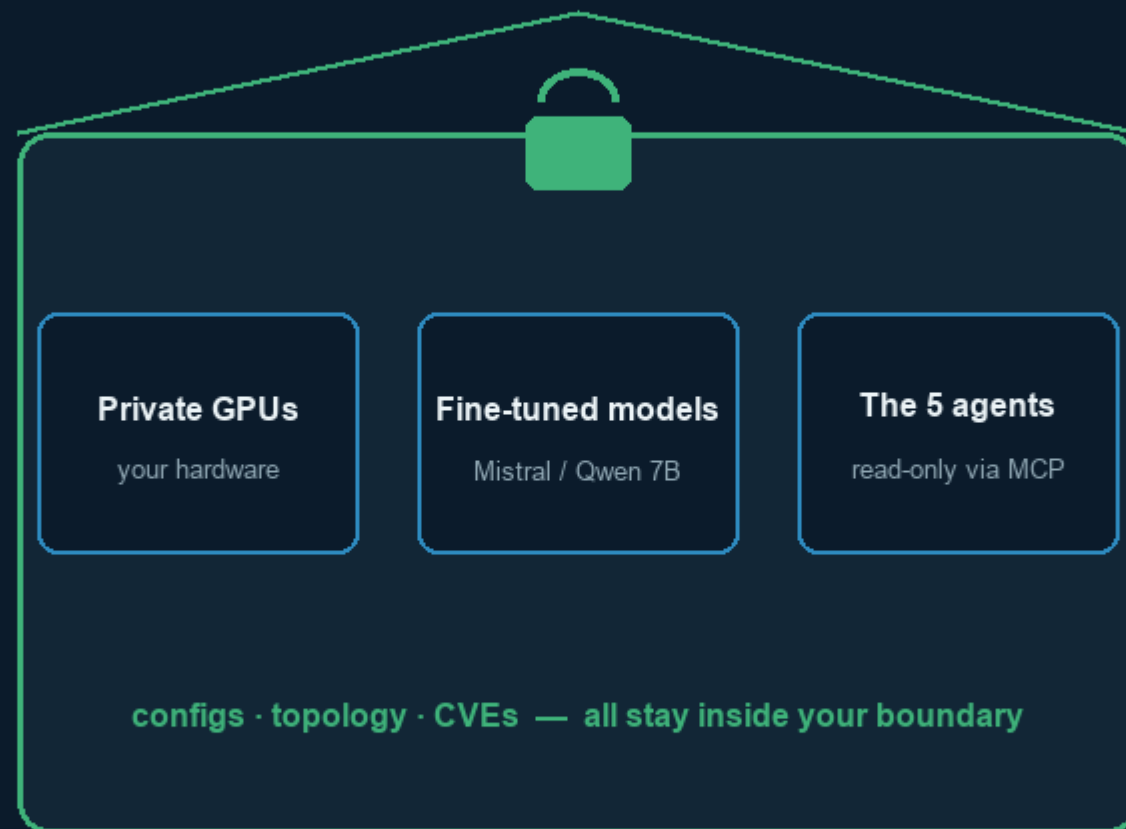


- The 4 specialists run self-hosted fine-tuned ~7B models on private GPUs — device configs never leave your boundary to a public API
- Every model we run — Stability = qwen2.5-7b · Troubleshooting = codellama-7b · Security = mistral-7b · Virtual Architect = va-v2 (LoRA on Qwen2.5-7B): four fine-tuned ~7B specialists, each on its own private GPU
- Plus the supporting cast — RAG embeddings = BAAI/bge-small-en-v1.5 (turns docs/cards into vectors for retrieval) · Team Leader = NO model at all (a deterministic router — code, not a guess) · Fallback = gpt-4o-mini, used only if a private endpoint is cold, and always logged (the card's 'Model:' line names which one actually ran)
- The Virtual Architect is the gated win: va-v2 LoRA on Qwen2.5-7B, promoted only after beating its base on a decontaminated benchmark
- The fine-tuning GATE is a hard firewall: a candidate must beat its base AND gpt-4o-mini on held-out data, or it does NOT ship
- Honest status: the VA cleared the gate and is deployed; the 3 diagnostic specialists serve weights not yet covered by a published gate report
- Transparent fallback: if a private endpoint is cold/down, the client falls back to gpt-4o-mini — and that's logged, not hidden

★ Private by default, and a fine-tune only ships if it provably beats the baseline.

Your data never leaves your building

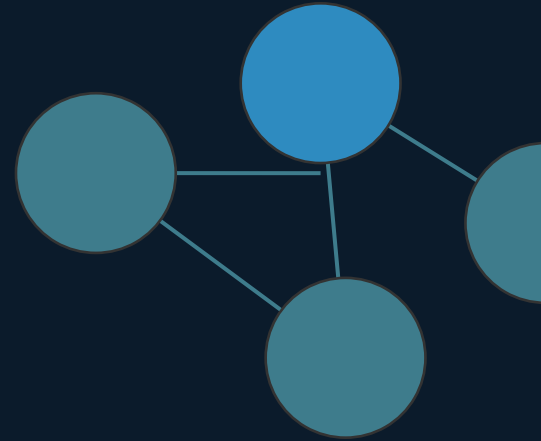
Self-hosted, fine-tuned models on your own GPUs



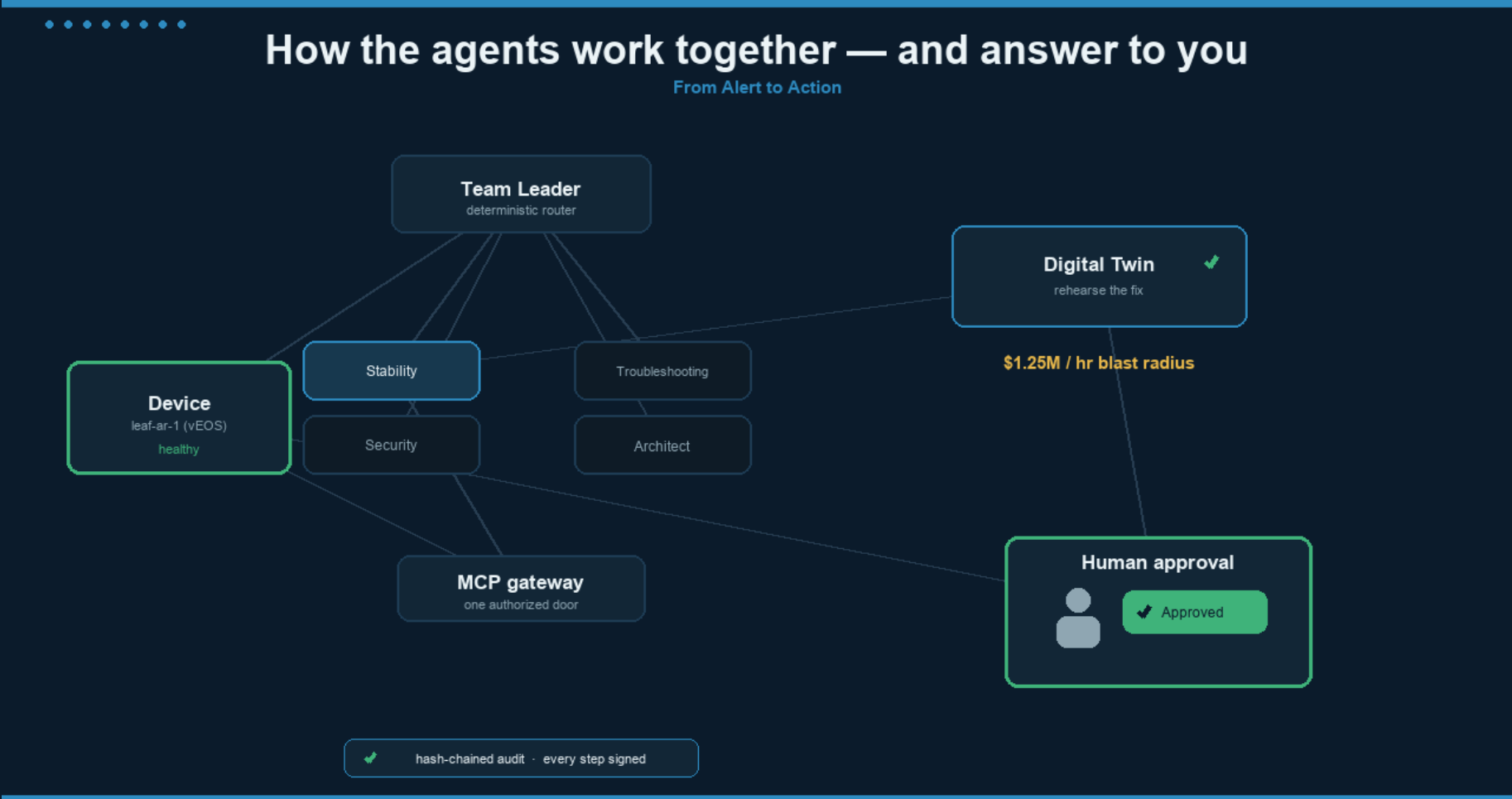
public LLM API
never touched

PART 3 · LIVE ON A REAL FABRIC

From alert to action — watch it happen



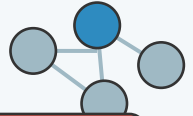
From alert to action – the 100-second story, once through



From alert to action — in about 100 seconds.

Alert → Team Leader → specialist (A2A) → live read (MCP) → digital twin → human approves → applied, with a signed audit trail.

Is the fabric healthy? — asked against 8 real devices



• LIVE

“Let's ask the question you'd ask at the start of any war-room — is the fabric healthy? — and watch it answer against eight real devices.”

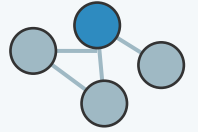
On screen

UI → Architect ReAct tab → “Is VXLAN EVPN healthy across all 8 fabric devices?”

On display

Fabric Health Card: BGP/EVPN session states, VTEPs, VNIs, severity-bucketed findings with evidence citations, and a \$/min blast radius.

★ It reads LIVE state, not a dashboard cache — and cites its evidence.



How 'is the fabric healthy?' is answered — live, not guessed

- **Intent → fast-path:** the question is recognised as a **fabric-health intent** and takes a **DETERMINISTIC code path** — it does NOT ask the LLM and it does NOT trust the cached Knowledge Graph
- **Live reads only:** it calls the read-only **evpn_parser MCP tool** once per EVPN node — the 6 EVPN speakers of the fabric (firewall + edge excluded as non-EVPN) — each call reaches the real box through the one MCP gateway and runs the BGP-EVPN summary
- **Deterministic parse:** code reads each neighbour state (Established = up · Idle/Active/Connect = down) plus the VNIs/VTEPs — the model never invents a status
- **Correlates both ends:** spine1 'down → 10.0.0.22' and leaf-nx-2 'down → 10.0.0.11' are recognised as ONE session seen from two sides → a single honest DEGRADED verdict (leaf-nx-2 still peers spine2, so the data plane survives on route-reflector redundancy)
- **Source:** adapter — a real live read at a real timestamp , not a dashboard cache, not a model guess

★ **Proof it's real:** shut ONE BGP neighbour on a leaf, ask again — in seconds it reports DEGRADED and names the exact down peer. It saw YOUR change on the live switch, not a cached picture.

"Is the fabric healthy?" — the live MCP round-trip

The question travels down to the **real switches** and the truth comes back up — **no cache, no model guess**.

▼ THE ASK & THE LIVE READ

1 "Is VXLAN-EVPN healthy?"
asked in the Architect ReAct tab

2 Fabric-health fast-path — a
deterministic route. **No LLM. No
cached graph**.

3 `evpn_parser(device)` x6 — a read-
only MCP tool call per EVPN node

4 MCP runs `show bgp l2vpn evpn
summary` over NX-API / SSH

L1

User Interface

Streamlit · Architect ReAct tab — where you ask the question

L3

Virtual Architect

picks the **fabric-health fast-path** · deterministic · grounded, never guesses

no LLM in this path

no cached KG

L4

MCP Server :8001

the **only thing allowed to touch hardware** · hosts the `evpn_parser` tool

L5

6 real EVPN switches

spine1 · spine2 · leaf-nx · leaf-nx-2 · leaf-ar · leaf-ar-2 (firewall/edge excluded)

▲ THE VERIFIED ANSWER

5 each switch returns its **real BGP-
EVPN neighbour states + VNIs**

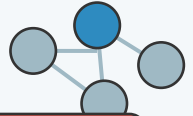
6 MCP parses it & stamps **source:
adapter** — proof it read the box

7 **HEALTHY / DEGRADED** verdict +
evidence citations + **\$ blast radius**

✓ reciprocal sessions correlated →
reported **once**, honestly

source: adapter = it read the actual switch — not a dashboard cache, not a model guess. Shut one neighbour → **DEGRADED** in seconds, exact peer named; **no shutdown** → back to **6/6 Established**.

Prove it's a graph, not a guess — watch the speed



• LIVE

“Now let me prove it's grounded in a graph, not guessing. Watch the speed.”

On screen

Type “Can leaf-nx reach leaf-ar?” then “Can leaf-nx-2 reach leaf-ar-2?”

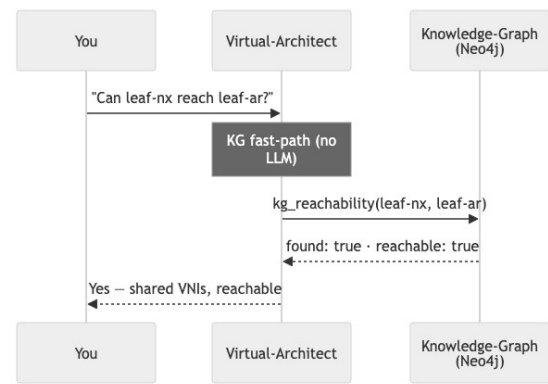
On display

First: Yes, ~0.01s — transit via spine1+spine2, shared VNIs. Second: No, ~0.13s — leaf-nx-2's BGP-EVPN sessions IDLE, overlay down.

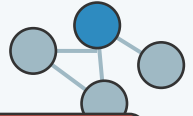
★ Sub-second graph-truth from Neo4j — not a model guess. Zero hallucination.

Demo 1B • Reachability — Knowledge-Graph lookup

kg-fast-path: VA → Neo4j Cypher. Sub-second graph truth, no device touched, zero hallucination.



An AI that says “I don't know”



• LIVE

“The thing I'm proudest of — an AI that knows what it doesn't know.”

On screen

Architect ReAct tab, 3 asks: (1) “Is leaf-nx-2 redundant to the spines?” (2) “Does it make sense to add a 3rd spine?” (3) “Best way to bake sourdough?”

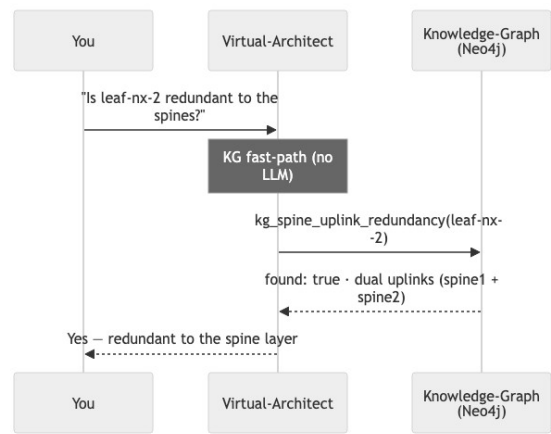
On display

(1) KG fast-path ~0.01s + IDLE-session caveat; (2) va-v2 design answer with a Sources: footer; (3) an honest ABSTAIN.

★ Anti-hallucination is the trust story. An AI that abstains is the only kind you put near production.

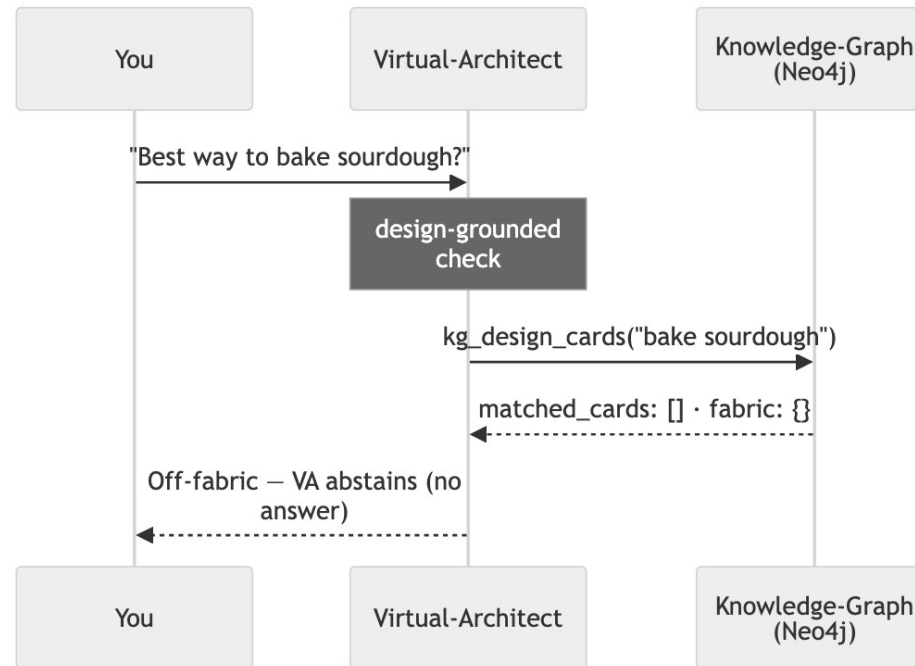
Design check — Knowledge-Graph (spine uplink redundancy)

kg-fast-path: grounded in the fabric graph — 2 distinct spine peers = redundant.

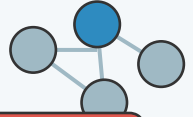


Grounded VA — abstains off-fabric

design-grounded: no matching fabric/KG cards → the model declines instead of hallucinating. 'An AI that says I don't know.'



Is the fabric compliant? — live, mapped to your framework



“The question your auditor asks — is this fabric compliant? Pick the framework — PCI DSS, or NIS2 for the EU — and watch it map live device facts to controls and the CVEs that actually touch what you run.”

On screen

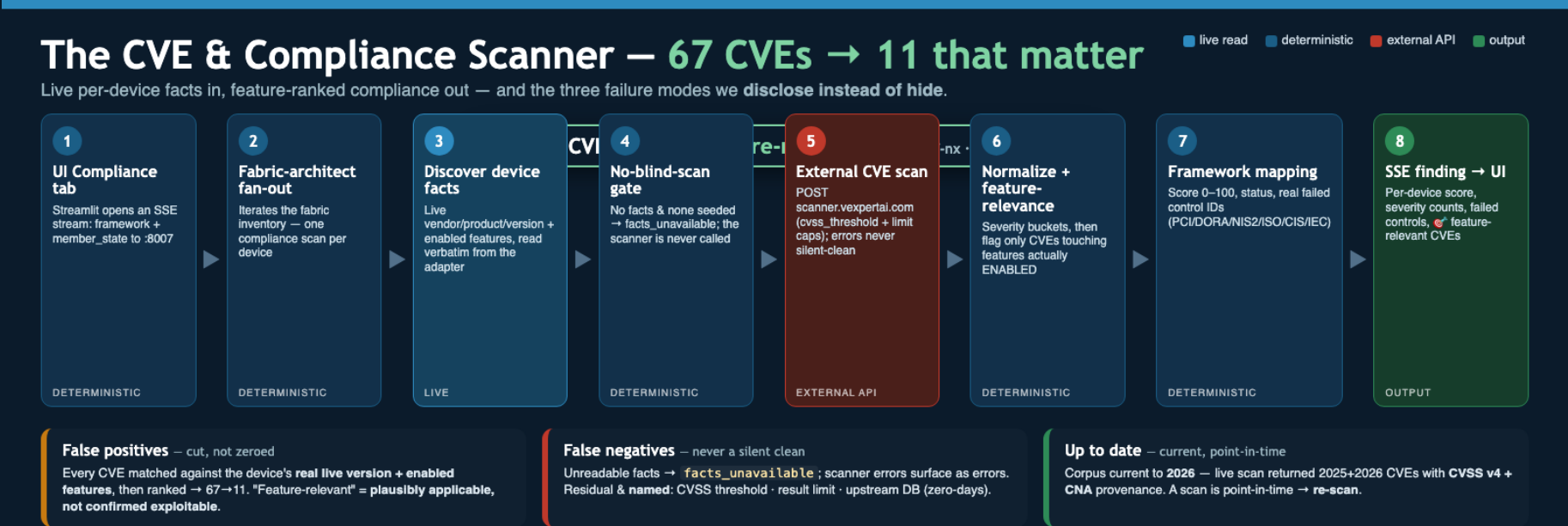
UI →  Compliance tab → Framework = PCI DSS v4.0 (or NIS2 (EU) → member state 'DE') → Check fabric compliance.

Audience sees

Per device, live: a compliance score + status (leaf-nx 37/100 NON_COMPLIANT), severity-bucketed CVE counts (1 critical • 20 high • 67 total), the FAILED control IDs (6.3.1, 6.3.3, 11.3.2...), and the  CVEs touching an ENABLED feature (CVE-2018-0313→NX-API, CVE-2024-20321→BGP).

★ Compliance from LIVE device facts, not a spreadsheet — and it knows which CVEs matter because it knows which features are ON.

How the scanner works — 67 CVEs → 11 that matter



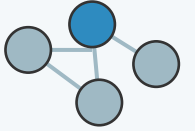
Inside the scanner — deterministic matching fused with hybrid RAG

VEXpertAI Vulnerability Advisor — how the scanner works

Deterministic CPE-based CVE matching fused with hybrid RAG + LLM advisories over a 240K+ CVE pgvector corpus — kept fresh by daily NVD + AI ingestion.



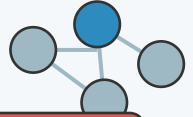
Deterministic CVE matching, then AI — over a ~300K-CVE corpus kept fresh daily



- **Two stages**, on purpose: **pure-code CPE matching FIRST** — walk the 3-level CPE tree, version-range gates, 0.95 confidence, NO LLM — THEN AI writes the advisory. The facts are deterministic; only the prose is generated.
- **Hybrid RAG**, not just vectors: pgvector ANN + Postgres full-text fused , then a cross-encoder re-ranks 50→5 — so the LLM sees the 5 best grounded chunks + the real vendor fix facts. (pgvector is a PostgreSQL extension that stores vector embeddings (a vector column type) and searches them by distance. ANN = Approximate Nearest Neighbor — instead of scanning every row to find the closest vectors it uses an index that returns the *probably*-closest ones much faster)
- **Provider-agnostic LLM**, fenced in: Gemini 2.5-flash (cascade) or Bedrock Claude Haiku 4.5; invented CVE IDs stripped, sources overwritten with real URLs, deterministic CVSS fallback if every model fails.
- Fresh by design: ~300K CVEs (1999→today); a daily NVD delta sync PLUS our own CPE backfill - NVD now enriches only ~15-20% of new CVEs (triage model, Apr 2026), so we cover what NVD does not; CVSS always refreshed v3.1→v4.0; CISA KEV daily.
- Accuracy is measured, not claimed: a weekly ground-truth harness fires an alert if recall drops below 95%. Honest: the text feature-match (0.6) and the AI CPE-backfill are best-effort.

★ The scanner behind DEMO 2 is your own open-source backend — and its design choice is the whole trust story: deterministic facts first, AI second, with the AI fenced so it can't invent a CVE or a source.

Live compliance, not a quarterly PDF



“Compliance isn't a quarterly spreadsheet anymore. Watch it scan all six devices against PCI in real time.”

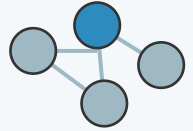
On screen

UI →  Compliance tab → scan all 6 devices vs PCI DSS (toggle a 2nd framework to show multi-standard).

Audience sees

4 Cisco NX-OS → PCI DSS 37 NON-COMPLIANT; 2 Arista → 100 COMPLIANT. Six frameworks available (PCI / DORA / NIS2 / ISO 27001 / CIS v8 / IEC 62443).

★ Live, multi-framework, vendor-aware compliance posture across the whole fabric.



Two stability fixes, rehearsed on the real-image twin

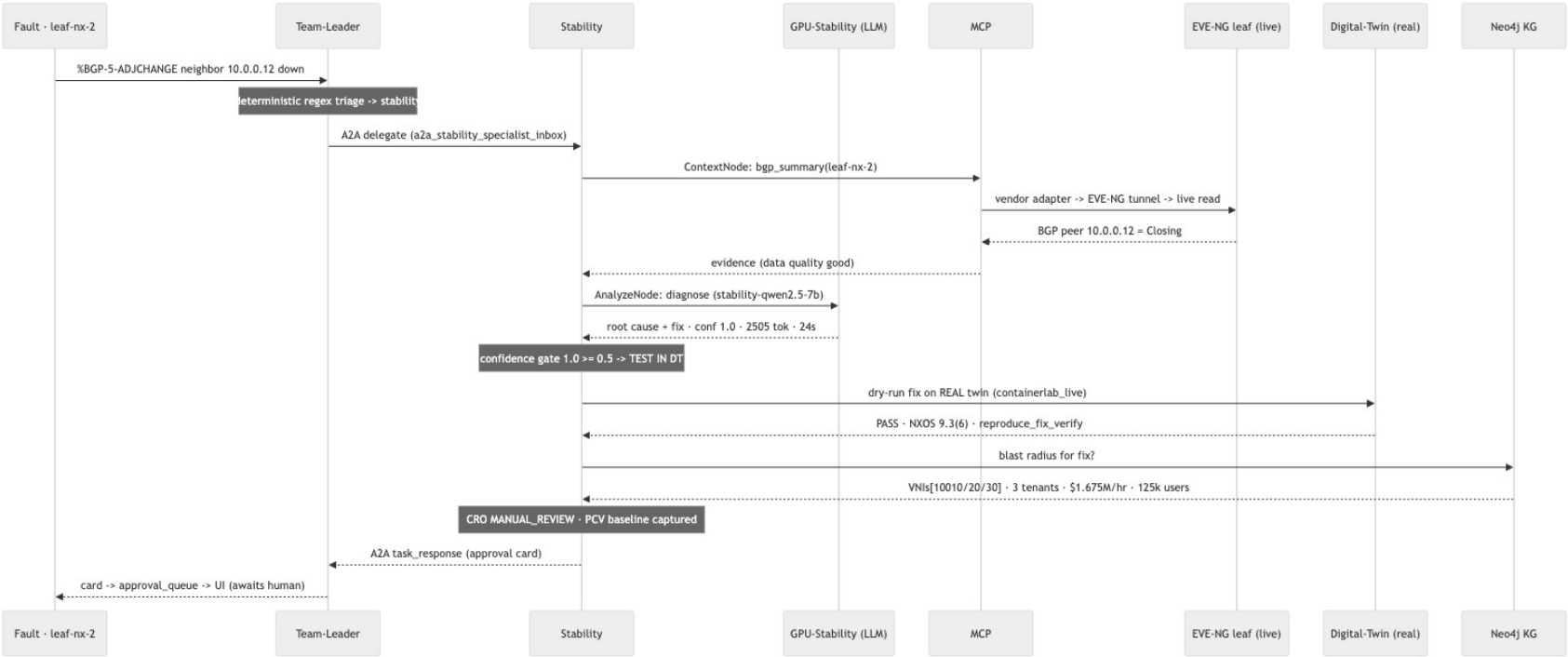
- **Use case 1** — BGP-EVPN peer admin-shut: shut a real peer on a leaf → agent diagnoses → fix 'neighbor X · no shutdown' → L1 graph-sim (reachability) → L2 metal twin REPRODUCES the shut, applies the fix, verifies `bgp_established_peers` recovers — on NX-OS 9.3(6). Proven passed.
- **Use case 2** — Interface admin-shutdown: shut a real interface (Eth1/3) → fix 'interface Eth1/3 · no shutdown' → L1 → L2 metal twin: interface up → admin-down → up, captured live on the real image. Proven passed.
- Both validate on the SAME OS as production via reproduce → fix → verify — the twin breaks it, fixes it, and proves recovery before a human approves. Not a graph guess.
- The honest rule: inject a REAL fault (a real shutdown), never a synthetic syslog — the agent grounds in LIVE evidence, so only a real fault gives the confidence to run the twin.
- Live commands + the full container-by-container data flow: [ppt/Stability-DT-UseCases-Runbook.md](#).

★ The differentiator made real: a fix isn't 'approved' until the SAME vendor OS as production has broken-fixed-verified it on the twin.

Stability Agent — incident → approval card (live, all agents)

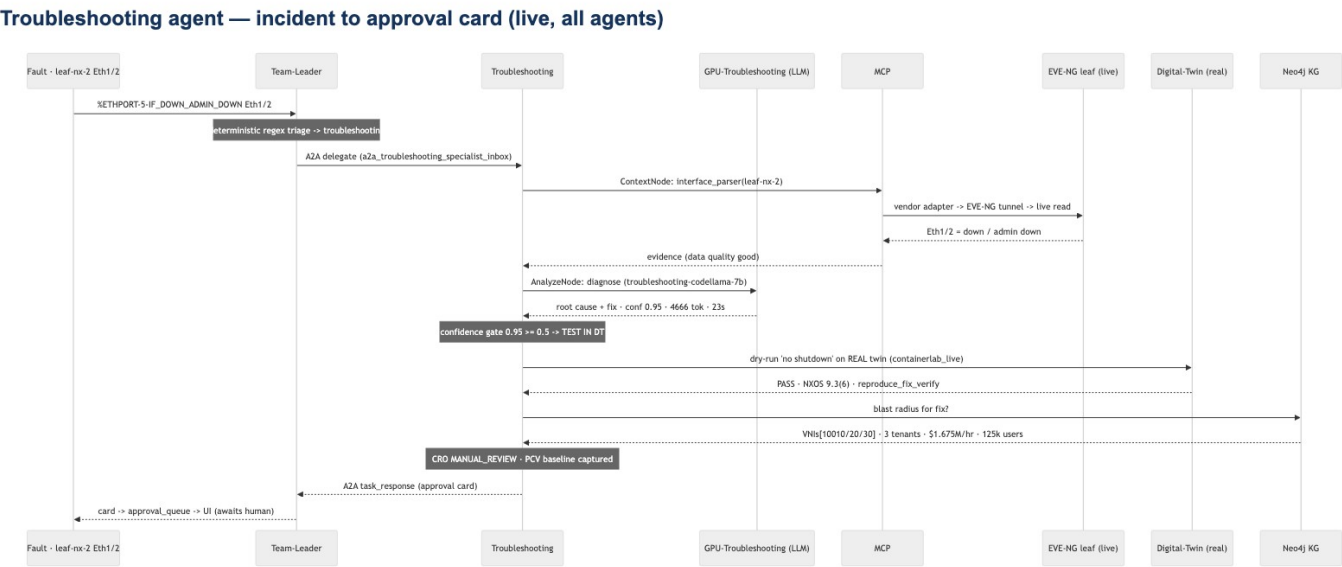
Real BGP-neighbor shut on leaf-nx-2 → TL triage → Stability(LLM stability-qwen2.5-7b) → live read → REAL twin L2 (containerlab) → CRO · Neo4j-KG blast radius · PCV → card

Stability agent — incident to approval card (live, all agents)



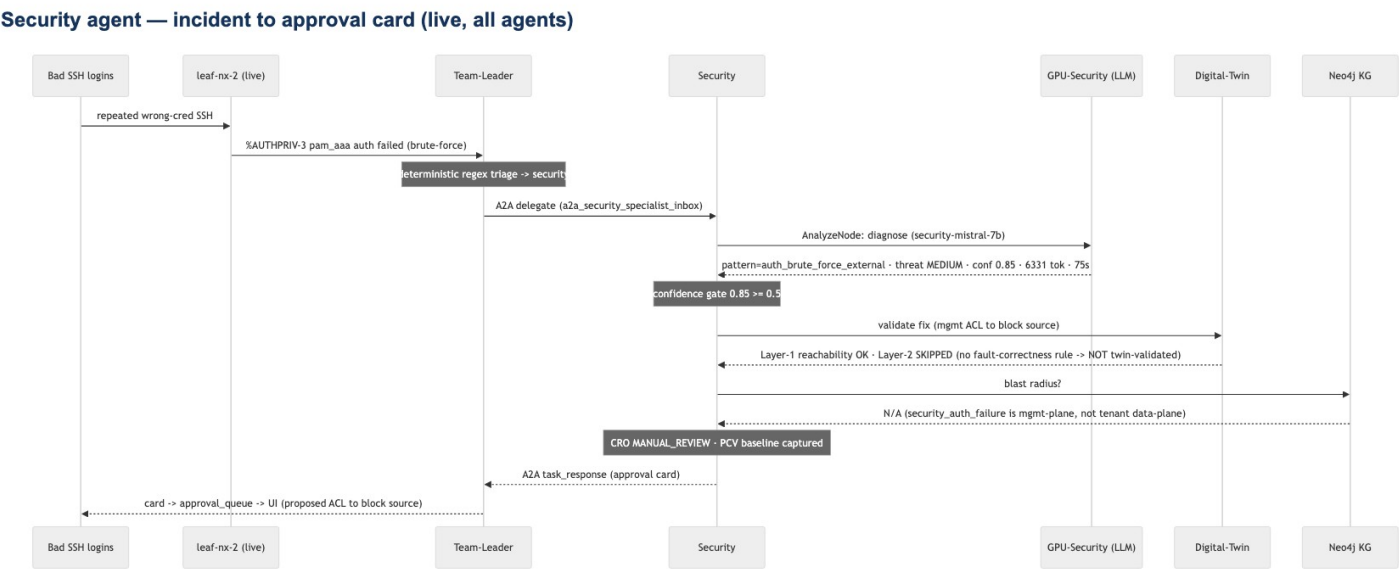
Troubleshooting Agent — incident → approval card (live, all agents)

Real Eth1/2 shut on leaf-nx-2 → TL triage → Troubleshooting(LLM troubleshooting-codellama-7b) → live read → REAL twin L2 → CRO · Neo4j-KG blast radius (\$1.675M/hr) · PCV → card



Security Agent — incident → approval card (live, all agents)

Brute-force SSH on leaf-nx-2 → TL triage → Security(LLM security-mistral-7b) → confidence gate → DT Layer-1 only (NOT twin-validated: no fault-correctness rule) → CRO · blast radius N/A (mgmt-plane) · PCV → card (ACL to block source)



A telnet config never reaches the switch



• LIVE

“And here's the seatbelt — a config with feature telnet never reaches the wire.”

On screen

Submit a proposed config containing 'feature telnet' to the change pipeline (strict mode).

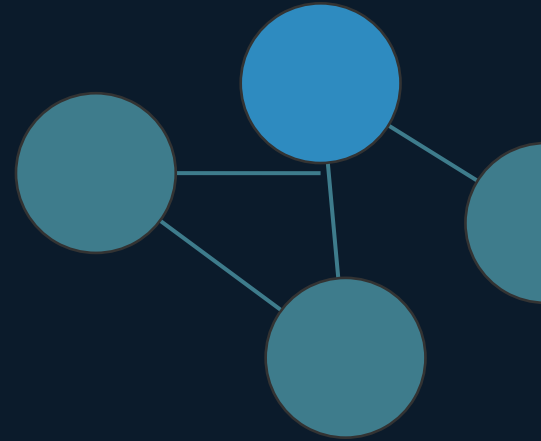
Audience sees

The CIS-mapped policy engine fires → raises a policy_violation INC → BLOCKS the apply before it reaches the switch.

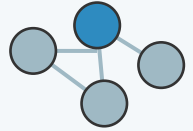
★ Agents propose, a human approves — and even then a policy engine is the last seatbelt.

PART 4 · WHY YOU CAN TRUST IT

The differentiators: it knows what it doesn't know



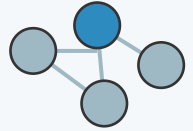
The AI is fenced — a human and a catalogue are in control



- Scenarios are DATA, not code: a YAML catalogue (scenarios + diagnostic trees + deterministic rules) defines triggers, routing, which MCP tools to call, and a confidence threshold
- Routing is deterministic regex on known syslogs — a rule sends 'RT mismatch' to Stability in ~3s; for a matched fault no model chose the route (an unmapped syslog falls back to an LLM classifier)
- The model is ONE step, gated on both sides: deterministic evidence-gathering before it; a confidence threshold + Digital Twin + policy gate after it
- Add a new fault = add a YAML scenario + tree. No code change, no redeploy of agent logic
- Specialists are pinned to a CLOSED pattern vocabulary — an unknown pattern can't be auto-approved

★ Here's the one thing that makes this safe enough to trust: the AI proposes, but it never decides — the catalogue and a human do.

Every decision is recorded, human-gated, and dollar-scoped



- Full audit trail: every decision is written to TimescaleDB (agent_reasoning_chains) — agent, confidence, status, the chain-of-thought, timestamped and replayable (signed-audit-event infra is in the schema; cryptographic chaining is on the roadmap)
- Human-in-the-loop is the real gate: nothing is applied until a human approves over ACP
- Change Risk Officer: deterministic policies (freeze windows, compliance zones, destructive-command checks) → ALLOW / MANUAL_REVIEW / BLOCK, each citing the policy that fired
- Blast Radius Forecaster: traverses the KG to turn a fix into business language — 'VNI 10010, 1 tenant, \$1.25M/hr, PCI-DSS'
- Executor guards: idempotency, command denylist, anti-storm, maintenance-window enforcement before anything reaches a device

★ Auditability is positive too — the chain records WHY a fix was ALLOWED, not just blocked. (Commands to show it live → next slide.)

Your network gets a seatbelt, an airbag, and a flight recorder



SEATBELT

Policy guardrail

A bad config — telnet, a permit-any rule — is blocked BEFORE it reaches the wire.



AIRBAG

Digital twin

Every proposed fix is rehearsed on a perfect copy before it ever touches prod.



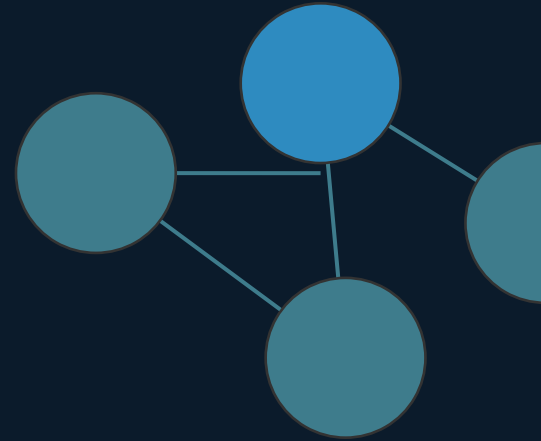
FLIGHT RECORDER

Audit trail

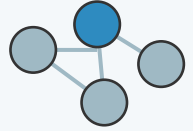
Every step — who asked, who approved, what changed — signed and hash-chained.

PART 5

Is it real? Does it scale? How do I adopt it?



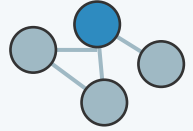
More devices, more faults, more vendors — mostly without code



- **More faults: write a YAML scenario + diagnostic tree.** No code, no redeploy — the catalogue IS the extension surface
- **More vendors:** a VendorAdapter ABC + registry already abstracts IOS-XE / NX-OS / Arista eAPI / Nokia SRL behind one seam
- **Vendor-agnostic plugin design** (VendorAdapter / DomainPack / InventorySource / Neutral Model) targets ~85-90% reuse across tenants/topologies
- **More compute:** each specialist runs on its own GPU — scale the fleet horizontally, per model, independently
- **Shared truth in the KG:** topology-as-code means a new device is declared, not hand-wired into agent logic

★ The cheap axis is faults and vendors (data + adapters); the expensive axis is concurrency.

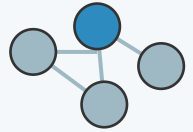
What it does NOT do yet — and where a human stays in the loop



- Demo-grade, not production-HA: single host, single Redis (no persistence/auth by default), one consumer per A2A queue — a crash can drop an in-flight message
- Some safety gates are advisory, not enforced: CRO BLOCK is displayed not hard-blocked, the Digital Twin is advisory — the human approval is the real enforced gate today
- KG sync is serial and silent: it drifts past ~7 real devices on a 14-device fabric, and stale data looks identical to fresh
- 7B model variance is real: small fine-tunes are sharper but narrower — abstention + the closed pattern vocabulary are the deliberate guardrails
- 'RAG' is curated-excerpt injection (not vector retrieval) on most paths; 3 of 4 specialist fine-tunes aren't yet gate-verified — the roadmap is written, the teeth are partly pending

★ The intent is real and well-designed; some of the teeth are still being fitted. That's the truth.

SaaS or on-prem / air-gapped — the models are yours to host



- Because the specialist models are self-hostable, the same stack runs SaaS (managed) or fully on-prem — no hard dependency on a public LLM
- Air-gapped / data-resident: run the GPU fleet + MCP gateway + KG + audit DB entirely inside your boundary; configs never egress
- Hybrid is the pragmatic default: private models for anything config-bearing, with a logged gpt-4o-mini fallback you can disable in strict mode
- Footprint today: ~15 Docker Compose services on one host + a small GPU box per specialist — easy to pilot; needs HA work before multi-tenant production

★ The self-hosted model is the unlock: strict on-prem is a config choice, not a re-architecture.

FROM ALERT TO ACTION

Useful beats impressive.

Clone the scaffold · wire your own specialist · reach us if it fits

github.com/eduardd76/autocon5-labs · [vExpertAI](#)

ed@vexpertai.com · [vexpertai.com](#)



Book a call · [vexpertai.com](#)